



Programmation java IV

Rapport de projet :
HelbRecrutezMoiSVP

Mr. Riggio

2025-2026

Table des matières

Introduction	3
Fonctionnalités de base	4
Gestion et interaction avec les candidats	4
Suivi des performances du recruteur	6
Gestion des modes de sélection des matchs	7
Gestion de la session de recrutement	8
Analyse et application des design patterns	9
Limitations	14
Conclusion.....	16
Accès LLM	16

Introduction

Dans le cadre du cours de Programmation Java IV, il nous a été demandé de développer une application proof of concept pour la société HELBÔBoulot. Cette application a pour objectif de former les recruteurs à la gestion et à la sélection de candidats. Elle simule des situations de recrutement au cours desquelles les utilisateurs doivent comparer différents profils et sélectionner celui présentant le score le plus élevé. Chaque choix influence le score de recrutement du recruteur au sein de sa session de formation. L'application a été conçue de manière flexible afin de permettre une variation du nombre de candidats proposés.

Le projet repose sur une architecture orientée objet développée en JavaFX, ce qui permet de structurer clairement l'application en séparant l'interface utilisateur, la logique métier et les différentes responsabilités des classes. Cette approche favorise la maintenabilité, la réutilisation du code et une meilleure organisation globale du système.

Pour la réalisation de cette application, des design patterns ont été utilisés afin d'assurer une bonne cohésion et un faible couplage entre les différentes classes, en s'appuyant sur les modèles étudiés au cours de Java IV. L'utilisation de Maven et de JUnit 5 était également requise : Maven a permis de gérer le projet et de générer un exécutable Java, tandis que JUnit 5 a été utilisé pour mettre en place des tests unitaires, notamment pour la vérification des CV des candidats.

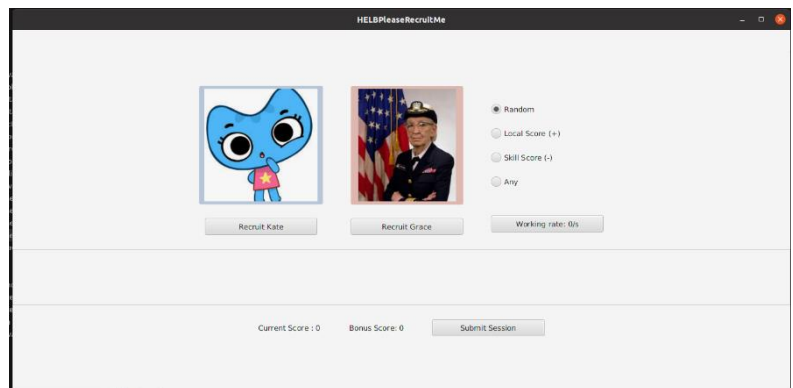
Parmi les fonctionnalités implémentées, l'utilisateur peut effectuer des matchs de sélection, choisir un candidat, consulter son profil, estimer son score, ainsi qu'accéder à l'historique des matchs. Il peut également visualiser son score, son score bonus et son working rate, puis clôturer sa session. L'ensemble des exigences du client a ainsi été couvert.

Ce rapport est structuré en plusieurs parties. La première présente les fonctionnalités de base de l'application. La deuxième est consacrée à l'analyse et à l'application des design patterns, en détaillant la structure de l'application. Une troisième partie aborde les limitations du projet. Enfin, le rapport se conclut par une synthèse des apprentissages réalisés ainsi que des fonctionnalités développées, et comprend également une section dédiée à l'accès au LLM.

Fonctionnalités de base

Gestion et interaction avec les candidats

Lors d'un match en cours, l'utilisateur peut gérer les candidats en consultant leur profil via un clic sur leur image. Il a ainsi accès à différentes informations, telles que la distance du candidat par rapport à l'entreprise ainsi que sa lettre de motivation. L'utilisateur peut également estimer le score du candidat afin d'obtenir un bonus, ou décider de le recruter en cliquant sur le bouton « Recruit NomCandidat ».



Les candidats sont générés dynamiquement au lancement de l'application et stockés dans une *ArrayList* d'objets *ApplicationProfile*. Cette structure contient l'ensemble des informations relatives à chaque candidat, notamment son nom, sa distance, sa lettre de motivation ainsi que son score. La création de ces objets est assurée par la fabrique *ApplicationProfileFactory*, utilisée dans la classe *TrainingSession* lors de l'instanciation de la session.

```
// Cette méthode envoie la liste de tout les profils de candidats valide.
public ArrayList<ApplicationProfile> createProfilesList()
{
    ApplicationProfile applicationProfile;
    // Récupère les données de chaque candidats valides dans une liste.
    ArrayList<ApplicationProfileData> applicationProfilesDatasList = ApplicationProfileService.createProfilesDatasList();
    // Initialisation d'une nouvelle liste pour stocker tout les profils de candidats.
    ArrayList<ApplicationProfile> profilesList = new ArrayList<>();

    // On créer par défaut un candidat de type alpha pour changer plus tard le type de profile.
    for (ApplicationProfileData applicationProfileData : applicationProfilesDatasList)
    {
        ApplicationProfile sourceProfile = new ApplicationProfileAlpha(
            applicationProfileData.getName(),
            applicationProfileData.getPicture(),
            applicationProfileData.getMotivation(),
            applicationProfileData.getDistance(),
            ApplicationProfileScoreCalculator.calculateDistanceScore(applicationProfileData.getDistance()),
            ApplicationProfileScoreCalculator.calculateMotivationScore(applicationProfileData.getMotivation())
        );

        // On verifie le type du profile avec le nom du candidat on change si besoin.
        if (ApplicationProfileTypeResolver.determineProfileType(applicationProfileData.getName()).equals(ApplicationProfileTypeResolver.PROFILE_TYPE_ALPHA))
        {
            applicationProfile = sourceProfile;
        }
        else
        {
            applicationProfile = new ApplicationProfileBeta(sourceProfile);
        }

        profilesList.add(applicationProfile);
    }

    return profilesList;
}
```

Classe *ApplicationProfileFactory.java* ligne 11

Cette partie du code montre que la factory s'appuie sur plusieurs classes pour créer correctement les profils candidats. La classe *ApplicationProfileService* lit les informations des candidats depuis le dossier DATA afin de créer une liste d'objets *ApplicationProfileData*. Ensuite, *ApplicationProfileValidator* vérifie si les données du candidat sont valides. Le score du candidat est calculé par *ApplicationProfileScoreCalculator*, tandis que *ApplicationTypeResolver* détermine si le profil est de type Alpha ou Beta. Une fois ces étapes terminées, l'objet *ApplicationProfile* est créé.

L'affichage des candidats dans l'interface est géré par la vue, qui crée un nombre de boutons correspondant au nombre maximal de candidats dans un match. Ces boutons sont stockés dans une *ArrayList* et transmis au contrôleur afin de gérer les événements. Deux *HashMaps* sont utilisées : la première associe les boutons permettant d'ouvrir le profil du candidat à son *ApplicationProfile*, et la seconde associe les boutons de sélection au candidat correspondant, afin de permettre soit la consultation du profil, soit la sélection du candidat lors d'un match. La vue met ensuite à jour l'affichage des informations via la méthode *updateProfiles*.

```
@Override
public void updateProfiles(Match match)
{
    ArrayList<ApplicationProfile> candidateList = match.getCandidatesProfilesList();

    profilesDetailsButtonMap.clear();
    profilesSubmitButtonMap.clear();

    for(int index = 0; index < candidateList.size(); index++)
    {
        ApplicationProfile candidate = candidateList.get(index);
        Image profileImage = new Image(getClass().getResource(candidate.getPicture()).toExternalForm());
        ImageView profileImageView = new ImageView(profileImage);
        profileImageView.setFitHeight(PROFILE_BUTTON_HEIGHT);
        profileImageView.setPreserveRatio(true);
        Button profileDetailsButton = profilesDetailsButtonList.get(index);
        Button profileSubmitButton = profilesSubmitButtonList.get(index);
        profileDetailsButton.setGraphic(profileImageView);
        profileSubmitButton.setText(SUBMIT_BUTTON_TEXT + candidate.getName());
        profilesDetailsButtonMap.put(profileDetailsButton, candidate);
        profilesSubmitButtonMap.put(profileSubmitButton, candidate);
    }
}
```

Classe *HELBRecruitMeView.java* ligne 243.java

Dans la méthode *updateProfiles*, on observe l'association des boutons avec les nouveaux candidats du match en cours, permettant ainsi la mise à jour dynamique de l'affichage des profils candidats.

Dans le contrôleur, lorsqu'un utilisateur clique sur le bouton d'ouverture du profil d'un candidat, la méthode *openCandidateProfile* est appelée avec le candidat en paramètre.

Celle-ci met à jour la vue en affichant les informations du candidat ainsi que son image et l'utilisateur peut aussi estimer le score dans une intervalle de 0 à 100 d'un candidat une fois par match avec une marge de 15% d'erreur pour obtenir 20% point bonus du delta match.

The screenshot shows a window titled "Kate profile" with a close button. The content includes:

- Kate -BETA-**
- Motivation :** Retrouvez mon CV sur <http://itsnotatrap.com>
- Distance :** 500
- Bonus :** Try to guess with 15.0% error margin to win 20.0% bonus
- Guess** The score value [0:100] with an input field containing the number 1.
- A "Confirm the guess" button.

Si l'utilisateur clique sur le bouton de sélection, le contrôleur appelle la méthode *confirmMatchEnd*, en lui passant en paramètre le candidat choisi ainsi que sa couleur. Cette méthode permet de valider la sélection, de préparer le choix d'un nouveau candidat et de mettre à jour le *ScrollPane* afin d'afficher l'historique des matchs.

Suivi des performances du recruteur

Le recruteur peut suivre ses performances dans la partie inférieure de l'application. Cette section affiche le score actuel de la session, le score bonus ainsi que le *working rate*, qui permet d'évaluer l'efficacité globale des décisions prises durant les différents matchs. Il est important de noter que le *working rate* n'influence pas le score final, mais constitue uniquement un indicateur de performance.

Current Score : 76

Bonus Score : 12

Working Rate : 0.25/s

Pour calculer le score de l'utilisateur, une variable score est utilisée. Celui-ci est mis à jour à la fin de chaque match via la méthode `calculateRecruiterScore` de la classe `TrainingSession`.

```
private void calculateRecruiterScore()
{
    this.score = 0;
    for (Match match : matchesList)
    {
        this.score += match.getDeltaMatch();
    }
}
```

Classe `TrainingSession.java` ligne 115

Cette méthode réinitialise d'abord le score du recruteur à 0, puis parcourt l'ensemble des matchs réalisés à l'aide d'un `foreach` sur la `matchesList`. Pour chaque match, elle incrémente le score en ajoutant le `deltaMatch`.

Le `deltaMatch` correspond à la différence entre le score du candidat sélectionné et celui du candidat non sélectionné, permettant ainsi de refléter la pertinence des choix effectués par l'utilisateur.

Le score bonus est stocké dans une variable nommée `bonusScore`. Il est incrémenté lorsque l'utilisateur fournit une estimation du score d'un candidat. Comme expliqué précédemment, cette estimation, comprise entre 0 et 100, ne peut être effectuée qu'une seule fois par match.

Lorsque l'utilisateur clique sur le bouton « confirm the guess », la méthode `calculateScoreBonus` de la classe `TrainingSession` est appelée avec en paramètre le score estimé. Le système récupère ensuite le `deltaMatch` du match en cours.

Une marge d'erreur de 15 % est appliquée afin de vérifier si l'estimation de l'utilisateur se situe dans un intervalle acceptable. Si tel est le cas, un bonus correspondant à 20 % du `deltaMatch` est ajouté au `bonusScore`. Dans le cas contraire, aucun bonus n'est attribué.

```
public void calculateScoreBonus(int estimatedScore){
    double allowedErrorPercent = BonusScoreConfig.ALLOWED_ERROR_PERCENT;
    double bonusPercent = BonusScoreConfig.BONUS_PERCENT;

    int candidateScore = this.viewedProfile.getScore();
    int deltaMatch = this.currentMatch.calculateDeltaMatchForBonusScore();

    int minEstimatedScore = (int) Math.round(candidateScore - (candidateScore * allowedErrorPercent));
    int maxEstimatedScore = (int) Math.round(candidateScore + (candidateScore * allowedErrorPercent));

    if (estimatedScore >= minEstimatedScore && estimatedScore <= maxEstimatedScore)
    {
        this.bonusScore += (int) Math.round(deltaMatch * bonusPercent);
        this.observer.updateRecruiterStatistics(this.score, this.bonusScore, this.workingRate);
    }
}
```

Classe `TrainingSession.java` 97

Dernièrement, le *working rate* est stocké dans une variable nommée *workingRate*. Il est mis à jour via une méthode appelée *calculateWorkingRate* de la classe *TrainingSession*, qui le calcule en divisant le nombre de matchs réalisés par le temps écoulé en secondes depuis le lancement de l'application.

Pour mesurer ce temps écoulé, un timer a été implémenté dans le contrôleur. Celui-ci appelle régulièrement une méthode de la session nommée *updateSessionElapsedTime*, permettant de maintenir à jour le temps total de la session et d'assurer un calcul précis du *working rate*.

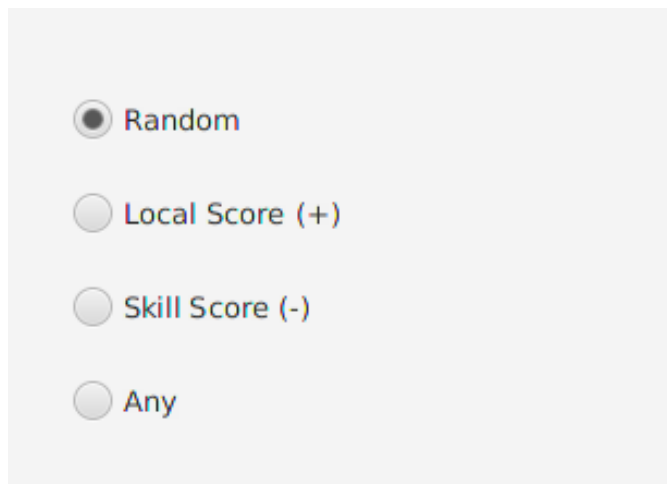
```
private void calculateWorkingRate()
{
    if(this.elapsedTime <= 0)
    {
        this.workingRate = 0;
        return;
    }
    double millisDivider = 1000.0;
    this.workingRate = this.matchesList.size() / (this.elapsedTime / millisDivider);
}
```

Classe *TrainingSession.java* ligne 127

L'affichage des statistiques est assuré par une méthode nommée *updateRecruiterStatistics* dans la vue. Cette méthode est implémentée via l'interface *HELBRecruitMePLZObserver*, conformément au principe selon lequel c'est le modèle qui notifie les changements. Ainsi, toute mise à jour des données entraîne automatiquement une actualisation de l'interface utilisateur.

Gestion des modes de sélection des matchs

L'utilisateur peut choisir le mode de sélection des matchs en cliquant sur un *radio button* correspondant au mode souhaité. Le mode *Random* génère des candidats de manière totalement aléatoire. Le mode *LocalScore* sélectionne des candidats en se basant sur la distance, en privilégiant ceux présentant le plus grand écart parmi les candidats non encore choisis. Le mode *SkillScore*, quant à lui, se base sur la lettre de motivation et sélectionne, parmi les candidats non encore choisis, ceux dont les profils présentent un faible écart. Enfin, le mode *Any* applique de manière aléatoire l'un des deux critères distance ou qualité de la lettre de motivation en sélectionnant également uniquement des candidats non encore choisis.



Le mode de sélection choisi est stocké dans une variable de type *ProfileSelectionStrategy*. Lorsque l'utilisateur sélectionne un autre *radio button*, le contrôleur met à jour la stratégie en appelant la méthode *setCurrentStrategy*, à laquelle il passe en paramètre le nom de la stratégie sélectionnée. Cette approche permet de modifier dynamiquement le

comportement de sélection des candidats en fonction du choix de l'utilisateur.

```
view.strategiesGroup.selectedToggleProperty().addListener((observable, oldValue, newValue) ->
{
    // Le null vu avec le prof
    if (newValue != null) {
        trainingSession.setCurrentStrategy(view.getProfileStrategy());
    }
});
```

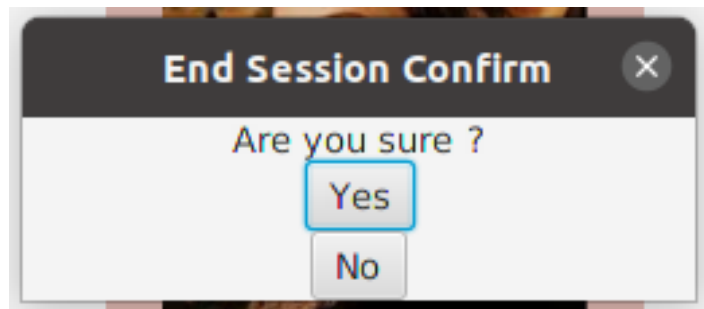
Classe HELBRecruitMeController.java ligne 52

Gestion de la session de recrutement

La classe TrainingSession permet de centraliser et de stocker l'ensemble des informations liées à la session du recruteur. Elle conserve en mémoire les matchs effectués, le score de l'utilisateur, le bonusScore, le workingRate, ainsi que le temps écoulé depuis le début de la session.

En plus de ce rôle de stockage, elle gère également la logique des matchs en générant de nouveaux candidats et en mettant à jour les données après chaque sélection effectuée par l'utilisateur.

L'utilisateur a la possibilité de clôturer sa session en cliquant sur le bouton « submit session ». Cette action déclenche l'ouverture d'une *confirm box* demandant la validation de la fin de session. Si l'utilisateur confirme, les données sont ensuite sauvegardées dans un fichier session.txt et affichées dans la console.

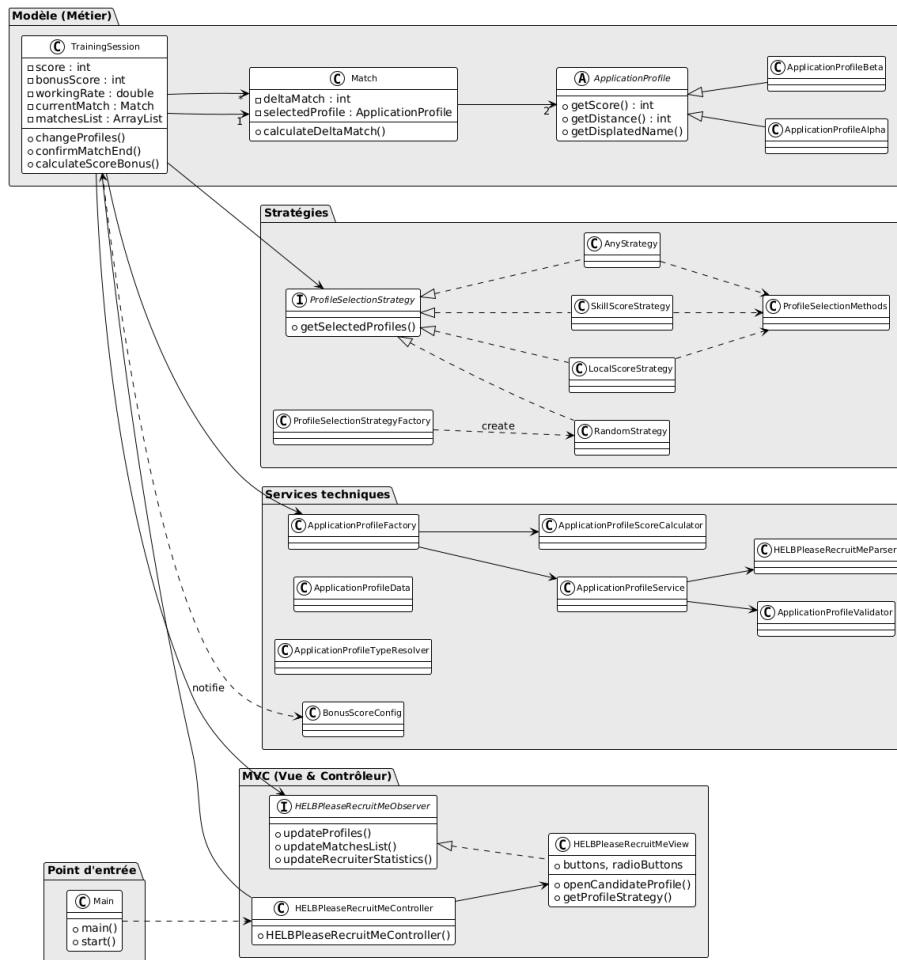


L'affichage commence par la liste des matchs réalisés : pour chacun, le nom du candidat gagnant ainsi que le deltaMatch sont indiqués. Ensuite, les statistiques globales de la session sont présentées, incluant le score de l'utilisateur, le bonusScore et le score final (correspondant à la somme du score et du bonus).

```
[INFO] -----
Successfully wrote to the file.
(Eve) Vs Roger : +65
(Cesar) Vs Grace : +7
Eve Vs (Bob) : -5
(Cesar) Vs Eve : -2
(Dave) Vs Alice : +11
-----
Current Score : 76
Bonus Score : 12
Final Score : 88
```

Enfin, la classe TrainingSession est transmise à un parseur chargé d'écrire l'historique de la session et de sauvegarder les informations importantes liées aux choix du recruteur.

Analyse et application des design patterns



Dans l'application, plusieurs design patterns ont été utilisés afin de structurer le code et éviter de concentrer toute la logique dans une seule classe. Le diagramme UML illustre clairement cette organisation. Chaque classe possède une responsabilité bien définie, ce qui rend le projet plus lisible, plus facile à modifier et plus simple à maintenir.

Tout d'abord, la classe `Main` permet de lancer l'application. Elle initialise le `HELBPleaseRecruitController`, qui joue un rôle central. En effet, le contrôleur fait le lien entre l'interface graphique et la logique métier. Lorsqu'un utilisateur interagit avec l'interface (par exemple en cliquant sur un bouton), la vue ne traite pas directement l'action : elle la transmet au contrôleur, qui appelle ensuite les méthodes appropriées dans `TrainingSession`. Cette organisation se rapproche du pattern MVC, car elle sépare clairement l'affichage, le contrôle et les données.

La partie **Model** regroupe les classes qui stockent les données de l'application. Par exemple, `ApplicationProfile` contient les informations d'un candidat (nom, image, motivation, distance, score). Les classes `ApplicationProfileAlpha` et `ApplicationProfileBeta` en héritent afin de représenter différents types de profils. La classe `Match` stocke les informations liées à un match, comme les candidats, le profil sélectionné, le score ou encore l'utilisation d'un

bonus. Enfin, `TrainingSession` représente l'état global de l'application, incluant le match en cours, l'historique, les scores et la stratégie utilisée.

Le pattern **Factory** est utilisé pour gérer la création des objets. La classe `ApplicationProfileFactory` permet de générer des profils sans exposer les détails de leur construction au reste du programme. Elle s'appuie sur plusieurs classes comme `ApplicationProfileService`, `ApplicationProfileScoreCalculator`, `ApplicationProfileTypeResolver` et `ApplicationProfileValidator`. Cette approche centralise la création des profils et facilite leur évolution : si un nouveau type de profil est ajouté ou si la logique change, il suffit de modifier la factory et les classes associées.

De la même manière, `ProfileSelectionStrategyFactory` permet de retourner la stratégie de sélection appropriée selon le contexte (aléatoire, basée sur le score local, basée sur les compétences, etc.). Cela évite de multiplier les conditions (if) dans `TrainingSession`.

Le pattern **Strategy** est implémenté via l'interface `ProfileSelectionStrategy`. Les classes `RandomStrategy`, `LocalScoreStrategy`, `SkillScoreStrategy` et `AnyStrategy` définissent chacune une manière différente de sélectionner un profil. Grâce à ce mécanisme, `TrainingSession` peut utiliser une stratégie sans connaître son implémentation concrète : elle appelle simplement la méthode définie par l'interface, et le comportement s'adapte dynamiquement.

Le pattern **Singleton** est utilisé pour certaines stratégies qui n'ont pas besoin d'êtreinstanciées plusieurs fois. La méthode `getInstance()` permet de réutiliser une unique instance, ce qui optimise l'utilisation de la mémoire et évite la création inutile d'objets.

Le pattern **Observer** permet de mettre à jour automatiquement l'interface lorsque l'état de `TrainingSession` change. L'interface `HELBPleaseRecruitObserver` permet à la vue d'être notifiée lors d'événements importants, comme la fin d'un match, la mise à jour des statistiques ou la création de nouveaux matchs. Cela évite de mélanger la logique métier avec l'affichage et améliore la séparation des responsabilités.

Enfin, le diagramme met en évidence plusieurs classes utilitaires. Par exemple, `ApplicationProfileScoreCalculator` gère le calcul des scores, `ApplicationProfileValidator` vérifie la validité des données, `ApplicationProfileTypeResolver` détermine le type de profil, et `HELBPleaseRecruitParser` s'occupe de la lecture et de l'écriture des fichiers. Ces classes spécialisées permettent d'éviter de surcharger des classes principales comme `TrainingSession` ou les factories.

En résumé, cette architecture repose sur une bonne séparation des responsabilités. Les modèles gèrent les données, les factories s'occupent de la création des objets, les stratégies définissent les comportements, l'observer gère la mise à jour de l'interface, et les classes utilitaires prennent en charge les traitements spécifiques. Même si cela augmente le nombre de classes, le code devient plus clair, plus flexible et plus facile à faire évoluer.

Après avoir expliqué l'architecture générale, il est utile de montrer quelques exemples concrets dans le code. Les exemples suivants ne reprennent pas tout le programme, mais montrent les parties les plus importantes des design patterns utilisés.

Le pattern Strategy est un des plus importants dans l'application, car il permet de changer la manière de sélectionner un profil sans modifier la classe TrainingSession.

```
public interface ProfileSelectionStrategy{
    public abstract ArrayList<ApplicationProfile> getSelectedProfiles(ArrayList<ApplicationProfile> profilesList);
}
```

Chaque stratégie implémente cette interface avec sa propre logique. Par exemple, RandomStrategy choisit un profil au hasard, tandis que SkillScoreStrategy et LocalScoreStrategy choisissent un profil selon un score précis.

```
package com.example;

import java.util.ArrayList;

// Cette stratégie permet de choisir deux profils au hasard.

public class RandomStrategy implements ProfileSelectionStrategy{
    private static RandomStrategy instance;

    private RandomStrategy()
    {}

    @Override
    public ArrayList<ApplicationProfile> getSelectedProfiles(ArrayList<ApplicationProfile> profilesList)
    {
        ArrayList<ApplicationProfile> selectedProfiles = new ArrayList<>();

        for(int counter = 0; counter < Match.MAX_CANDIDATES_NUMBERS; counter++)
        {
            int profileIndex = (int)(Math.random() * profilesList.size());

            while (selectedProfiles.contains(profilesList.get(profileIndex)))
            {
                profileIndex = (int)(Math.random() * profilesList.size());
            }

            selectedProfiles.add(profilesList.get(profileIndex));
        }

        return selectedProfiles;
    }

    public static RandomStrategy getInstance() {
        if (instance == null) {
            instance = new RandomStrategy();
        }
        return instance;
    }
}
```

Le Singleton est utilisé dans certaines stratégies pour éviter de créer plusieurs fois le même objet. Comme les stratégies ne stockent pas forcément beaucoup d'informations différentes, une seule instance suffit.

Comme on peut voir dans la stratégie Random, un Singleton possède un constructeur en privé, ce qui empêche de créer des objets directement avec new. Il possède aussi une variable statique appelée instance qui stocke l'unique objet de la classe. Cette instance est récupérée grâce à la méthode getInstance(), qui crée l'objet une seule fois puis le réutilise ensuite.

Le pattern Factory est utilisé dans deux endroits de l'application : pour les profils et pour les stratégies. Pour les profils, ApplicationProfileFactory s'occupe de créer les objets ApplicationProfile avec les bonnes données, les bons scores et le bon type. Pour les

stratégies, ProfileSelectionStrategyFactory retourne la stratégie demandée selon le choix de l'utilisateur ou du programme.

L'avantage est que la logique de création est centralisée dans une seule classe. Comme ça, TrainingSession n'a pas besoin de savoir comment créer les profils ou les stratégies. Elle demande juste l'objet dont elle a besoin. Cela rend le code plus propre et plus facile à modifier.

Dans la suite, on montre surtout un exemple avec la factory des stratégies, car elle est plus simple à comprendre et elle montre bien le principe du pattern Factory.

```
package com.example;

// Cette classe joue un rôle pour créer les stratégies de sélection de profils des candidats en fonction des choix du recruteur.

public class ProfileSelectionStrategyFactory{
    public static final String RANDOM = "Random";
    public static final String LOCAL_SCORE = "LocalScore";
    public static final String SKILL_SCORE = "SkillScore";
    public static final String ANY = "Any";

    public static ProfileSelectionStrategy createStrategy(String strategyChoice)
    {
        switch (strategyChoice) {
            case RANDOM:
                return RandomStrategy.getInstance();
            case LOCAL_SCORE:
                return LocalScoreStrategy.getInstance();
            case SKILL_SCORE:
                return SkillScoreStrategy.getInstance();
            case ANY:
                return AnyStrategy.getInstance();
        }
        return RandomStrategy.getInstance();
    }
}
```

Dans cet exemple, la factory reçoit un type de stratégie et retourne l'objet correspondant. On remarque aussi que les stratégies sont récupérées avec getInstance(), car elles utilisent le pattern Singleton. Cela évite de recréer une nouvelle stratégie à chaque appel.

Le pattern Observer est utilisé pour mettre à jour l'interface quand l'état de la session change. Par exemple, quand un match se termine ou quand les statistiques des recruteurs sont recalculées, la vue doit être informée.

```
package com.example;

import java.util.ArrayList;

// Cette interface permet de mettre à jour l'interface du recruteur
// lorsqu'il y a des changements dans les profils des candidats,
// dans les matches ou dans les scores.
// Elle suit le principe de l'Observer pour notifier la vue
// des modifications effectuées dans la session.

public interface HELBPleaseRecruitMeObserver {
    public abstract void updateProfiles(Match match);

    public abstract void updateMatchesList(ArrayList<Match> matchesList);

    public abstract void updateRecruiterStatistics(int score, int bonusScore, double workingRate);

    public abstract void onNoCandidatesAvailable();
}
```

Dans TrainingSession, on garde une référence vers un observer. Comme dans notre application il n'y a qu'une seule vue principale, on n'a pas besoin d'utiliser une ArrayList d'observers. On stocke donc directement l'observer dans une variable. Quand l'état de la session change, par exemple après un match ou après la mise à jour des statistiques, TrainingSession appelle les méthodes de l'observer pour prévenir la vue.

L'extrait de code ci-dessous illustre cette communication entre la session et l'observer, où plusieurs méthodes sont appelées pour mettre à jour les différentes parties de l'interface (profils, liste des matchs et statistiques).

```
public void confirmMatchEnd(ApplicationProfile selectedProfile, String color)
{
    this.currentMatch.setSelectedProfile(selectedProfile);
    this.currentMatch.calculateDeltaMatch();
    this.currentMatch.setMatchColor(color);
    this.matchesList.add(currentMatch);
    changeProfiles();
    updateRecruiterStatistics();
    // Mise à jour de la vue.
    this.observer.updateMatchesList(matchesList);
    this.observer.updateRecruiterStatistics(this.score, this.bonusScore, this.workingRate);
}
```



Le diagramme de séquence ci-dessus montre le déroulement des principales interactions dans l'application, depuis le lancement jusqu'à la fin de session. On y voit clairement le rôle de chaque composant (Vue, Contrôleur, TrainingSession, Stratégie, Parser) et les échanges entre eux. Il illustre aussi comment les actions de l'utilisateur (consultation d'un profil, calcul de bonus, recrutement) déclenchent des traitements dans la logique métier, puis des mises à jour de l'interface via l'observer. Cela permet de mieux comprendre le fonctionnement global du système et la collaboration entre les différentes classes.

Limitations

Même si l'application fonctionne correctement et permet de simuler des situations de recrutement, plusieurs limitations peuvent être observées. Ces limites concernent principalement le nombre de candidats, la gestion des données, l'interface ainsi que certains choix techniques qui réduisent la flexibilité de l'application sur le long terme.

La première limitation importante concerne le nombre de candidats disponibles. Environ 12 profils sont présents dans le dossier DATA, mais tous ne sont pas nécessairement exploitables. Certains candidats peuvent être rejetés par *ApplicationProfileValidator* lorsque leurs données sont incomplètes ou incorrectes. Dans le cas présent, 2 candidats ont été retirés, ce qui réduit encore le nombre réel de profils disponibles pour les matches.

Ce phénomène devient rapidement visible lorsque plusieurs matches sont enchaînés. La liste des candidats se vide progressivement, en particulier lorsque le mode sélectionné n'est pas *Random*. Les modes comme *LocalScore*, *SkillScore* ou *Any* reposent sur des critères précis pour sélectionner les profils. Comme certains candidats ont déjà été utilisés ou rejetés, le nombre de profils disponibles diminue progressivement au fil de l'utilisation.

Cette situation est encore plus marquée lorsque le nombre de candidats affichés par match est augmenté. Plus de profils sont présentés simultanément, plus la liste globale se réduit rapidement. Cela montre que l'application n'est pas totalement adaptée à des sessions prolongées ou à un volume important de candidats.

Une autre limitation concerne la gestion des couleurs dans l'interface. Les couleurs sont stockées dans une variable *HEX_COLORS*, limitée à 4 valeurs. Cela signifie que si davantage de candidats doivent être affichés, le code doit être modifié manuellement. Ce système apparaît donc comme peu flexible et dépend fortement des choix techniques initiaux.

Le système de score constitue également une limite. Les scores sont compris entre 0 et 100, ce qui simplifie le fonctionnement, mais réduit la précision de l'évaluation. Une distinction détaillée entre plusieurs critères comme les compétences, la motivation, l'expérience ou la distance n'est pas possible. L'évaluation des candidats reste donc relativement globale et parfois simplifiée.

Une autre limite importante provient du stockage des candidats dans les fichiers du dossier DATA. Le contenu de l'application est ainsi rendu statique. Pour qu'un candidat soit ajouté, modifié ou supprimé, une intervention directe dans les fichiers est nécessaire. La création de nouveaux profils par l'utilisateur depuis l'application n'est donc pas possible, ce qui limite l'évolution du contenu et peut rendre l'expérience répétitive après plusieurs utilisations.

L'application dépend également fortement de la qualité des données. Lorsqu'un fichier candidat est mal formaté ou incomplet, il peut être rejeté par le validateur. Ce mécanisme permet d'éviter certaines erreurs, mais contribue également à réduire le nombre de candidats réellement exploitables lorsque le format attendu n'est pas respecté.

Au niveau de l'interface, certaines limites apparaissent lorsque plusieurs candidats sont affichés simultanément. Les images, les informations et les boutons occupent un espace

important, ce qui peut rendre l'affichage plus chargé et moins lisible. Une meilleure organisation visuelle serait nécessaire afin de préserver une expérience utilisateur optimale.

Enfin, le *workingRate* est calculé en fonction du temps et du nombre de matchs réalisés. Cet indicateur mesure principalement la rapidité d'utilisation, mais la qualité des décisions prises n'est pas réellement prise en compte. Une vision partielle des performances est donc fournie.

Plusieurs améliorations peuvent être envisagées afin de corriger ces limitations. Tout d'abord, le nombre de candidats présents dans le dossier DATA pourrait être augmenté afin de prolonger la durée d'utilisation avant épuisement des profils disponibles. Il serait également important de vérifier que les fichiers soient correctement formatés afin de limiter les rejets.

Une amélioration importante consisterait à permettre la création de candidats directement depuis l'application. Les informations nécessaires (nom, image, distance, motivation, etc.) pourraient ainsi être ajoutées par l'utilisateur, ce qui rendrait le système plus dynamique et évolutif.

La gestion des couleurs pourrait également être améliorée grâce à une génération automatique des couleurs selon le nombre de candidats affichés, au lieu de s'appuyer sur une liste fixe. Une plus grande flexibilité de l'interface serait ainsi obtenue.

Le système de score pourrait être enrichi par l'introduction de plusieurs critères, comme les compétences, la motivation ou l'expérience, afin de permettre une évaluation plus précise et plus réaliste des candidats.

Les modes de sélection pourraient également être renforcés afin de mieux s'adapter aux variations du nombre de profils disponibles. Une gestion plus souple des candidats déjà utilisés pourrait être envisagée afin d'améliorer la continuité de l'expérience.

L'interface pourrait aussi être optimisée grâce à une grille, un système de défilement ou une mise en page responsive, afin de conserver une bonne lisibilité même lorsqu'un grand nombre de candidats est affiché.

Enfin, le *workingRate* pourrait être amélioré en intégrant également la qualité des décisions prises, et non uniquement la rapidité d'exécution.

En conclusion, les principales améliorations concerneraient l'augmentation du nombre de candidats, une plus grande dynamique du contenu, une amélioration de l'interface, un enrichissement du système de score ainsi qu'une meilleure flexibilité des modes de sélection. Ces évolutions permettraient de rendre l'application plus stable, plus évolutive et plus agréable à utiliser sur le long terme.

Conclusion

Ce projet m'a permis de développer une application de simulation de recrutement en Java tout en appliquant concrètement les notions vues au cours. J'ai pu implémenter plusieurs fonctionnalités importantes comme la gestion des candidats, le système de score, le *working rate* et l'historique des matchs.

La mise en place de design patterns comme *Factory*, *Strategy* et *Observer* m'a permis de mieux comprendre leur utilité dans l'organisation et la modularité du code. L'utilisation de Maven et de JUnit m'a également renforcé dans l'organisation du projet et l'importance des tests.

Même si certaines améliorations restent possibles, comme une interface plus flexible ou un système d'évaluation plus complet, ce projet a été une expérience très enrichissante. Il m'a permis de renforcer mes compétences en Java, en programmation orientée objet et en conception d'applications.

Accès LLM

Un compte ChatGPT a été créé conformément aux consignes données par le professeur. Cet outil a été utilisé comme support d'aide tout au long du projet, principalement pour la rédaction du rapport, l'amélioration des commentaires des classes et des méthodes, la proposition de noms de variables plus explicites, ainsi que pour suggérer des pistes de correction ou d'amélioration de certaines parties du code.

Son utilisation est restée ponctuelle, réfléchie et critique. Les propositions fournies ont systématiquement été analysées, testées et adaptées en fonction des besoins du projet. ChatGPT n'a pas été utilisé pour générer automatiquement l'intégralité du code, mais comme un outil d'assistance à la réflexion, au débogage et à la clarification de certaines implémentations techniques.

L'identifiant du compte ainsi que les discussions demandées sont fournis en annexe.

Identifiant du compte :

Email : ucar.ali.java.2526@gmail.com

Mot de passe : *AliHelb1234!*

5 dernières discussions :

<https://chatgpt.com/share/6a08a1f3-953c-8327-966f-ec9b2b07a40b>

<https://chatgpt.com/share/6a08a1d2-32d0-8328-8087-90c3179e9aa4>

<https://chatgpt.com/share/6a08a1c4-d8ac-8328-a0e9-7031f8372849>

<https://chatgpt.com/share/6a08a1b5-a684-838f-91a6-52fe1ed03090>

<https://chatgpt.com/share/6a08a183-f474-8332-896d-fe0662ebb9c8>